

Dataproc: Cluster vs Serverless Batches

Arquitetura Pub/Sub → Cloud Run → Dataproc

Comparação de Características

Critério	Dataproc Cluster	Serverless Batches
INFRAESTRUTURA		
Provisionamento	Manual — define master, workers, tipo de máquina. Tempo ~2–5 min.	Automático. Nenhum cluster para gerenciar. Inicia em ~30–60 s.
Ciclo de vida	Persistente ou efêmero. Requer gestão ativa.	Totalmente gerenciado. Infra destruída ao fim do batch.
Rede / VPC	Suporte completo a Shared VPC, subrede customizada, Private Google Access.	Suporta Shared VPC. Requer subnetwork explícita.
Customização SO	Alta — init actions, imagens custom, pacotes via pip/apt.	Limitada — via custom container (Docker) ou dependências no config.
CUSTO		
Modelo de cobrança	Por VM ativa (vCPU + memória + disco). Cobra mesmo ocioso.	Por DCU (Dataproc Compute Unit) consumida. Paga só o que roda.
Workloads esporádicos	Ineficiente — cluster parado gera custo. Mitigado com clusters efêmeros.	Ideal — zero custo quando não há batches rodando.
Workloads contínuos	Mais eficiente — custo previsível, overhead amortizado.	Pode ser mais caro — overhead por DCU em jobs muito frequentes.
PERFORMANCE		
Autoscaling	Via Autoscaling Policies (YARN metrics). Requer tuning.	Automático e transparente. GCP escala sem configuração.
Latência de startup	2–5 min para cluster frio. Persistentes têm latência zero.	30–90 s por batch. Sem opção de manter quente.
Jobs interativos	Suportado — Jupyter/Workbench, SSH, Spark shell.	Não suportado — apenas jobs batch assíncronos.
OPERAÇÃO		
Terraform	google_dataproc_cluster — recurso estável e maduro.	google_dataproc_batch — disponível, menos parâmetros expostos.
Cloud Composer	CreateCluster + SubmitJob + DeleteCluster operators (3 tasks).	DataprocCreateBatchOperator — 1 task por job.
IAM / SA	SA do cluster precisa de roles em Compute, Storage etc.	SA de runtime do batch. Requer roles/dataproc.worker + acesso aos recursos alvo.
Observabilidade	Spark History Server nativo. YARN UI disponível.	Spark History Server gerenciado automaticamente pelo GCP.

Recomendação Geral

Para pipelines orientados a eventos (Pub/Sub → Cloud Run → Dataproc), o **Serverless Batches** é a escolha natural: cada evento dispara um batch isolado, sem cluster pré-aquecido, e o custo é zero entre execuções. Reserve clusters para desenvolvimento interativo (Jupyter/Workbench) ou workloads contínuos onde o overhead de startup seria amortizado.

Padrão recomendado: WorkflowTemplate configurado via portal do GCP (futuramente via módulo Terraform). Cloud Run referencia apenas o template ID — toda a config de infra fica centralizada no template.

Arquitetura: Pub/Sub → Cloud Run → Dataproc Serverless

O fluxo de eventos passa por três camadas principais, cada uma com responsabilidades distintas:

Pub/Sub	Entrega at-least-once. O message_id do evento deve ser usado como requestId determinístico para garantir idempotência na chamada ao WorkflowTemplate.
Cloud Run	Consome o evento, valida, e chama instantiateWorkflowTemplate passando apenas o template ID e parâmetros dinâmicos do evento. Roda com SA próprio (roles/dataproc.editor).
WorkflowTemplate	Criado e gerenciado via portal do GCP (futuramente via Terraform). Centraliza toda a config do batch: runtime SA, subrede, versão Spark, job principal, dependências.
Serverless Batch	Job Spark isolado por evento, instanciado a partir do template. Infra provisionada e destruída automaticamente pelo GCP ao fim da execução.
Shared VPC	Configuração de subnetwork e Private Google Access definidas dentro do WorkflowTemplate. Não requer config de rede no payload do Cloud Run.

Decisões de Design

WorkflowTemplate	A configuração do batch (runtime, SA, subrede, dependências, job principal) fica centralizada no template gerenciado via portal do GCP. O Cloud Run referencia apenas o template ID, sem embutir config de infra no código.
Fire-and-forget vs. Polling	O Cloud Run chama instantiateWorkflowTemplate e pode retornar 200 imediatamente. Se o job demorar mais que 60 min (timeout máx do Run), use Workflows ou outro tópico Pub/Sub para polling externo.
Idempotência	Use requestId determinístico baseado no message_id do Pub/Sub na chamada de instantiateWorkflowTemplate. Evita disparar dois jobs para o mesmo evento em cenários de at-least-once delivery.
SA Chain	Cloud Run SA → roles/dataproc.editor para instanciar o template. O SA de runtime do job é definido dentro do próprio WorkflowTemplate no portal.
Shared VPC	Subnetwork e configurações de rede são definidas no WorkflowTemplate. Não há necessidade de passá-las no payload do Cloud Run. Private Google Access obrigatório na subrede.
Evolução futura	Quando o módulo Terraform de WorkflowTemplate estiver pronto, a migração é transparente para o Cloud Run — o ID do template muda, o código não.